Assignment No. 1

Title:
To Study Layers in Cloud Architecture, Cloud Service Models, and Cloud Deployment Models with Respect to Different Service Providers such as Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform (GCP), and others.

Objective:

The objective of this assignment is to understand and analyze the layers of cloud architecture, cloud service models (IaaS, PaaS, SaaS), and cloud deployment models (Public, Private, Hybrid, Multi-cloud). The study will focus on comparing and contrasting these aspects among leading cloud service providers such as Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform (GCP), and others.

Theory:

Part 1: Study of Cloud Architecture Layers

1. Research and document the various **layers in cloud architecture**, including:
    - Physical Layer (Infrastructure)
    - Virtualization Layer
    - Infrastructure Layer
    - Platform Layer
    - Application Layer
    - Management and Security Layers
2. Compare how AWS, Azure, and GCP implement these layers in their respective architectures.
3. Create a diagram or chart to visually represent the cloud architecture layers for each provider.

Part 2: Study of Cloud Service Models

1. Explore the three primary **cloud service models** (IaaS, PaaS, SaaS):
    - Define and explain each model with examples.
    - Identify the specific services offered by AWS, Azure, and GCP under each model.
        - **IaaS Examples:** AWS EC2, Azure Virtual Machines, Google Compute Engine.
        - **PaaS Examples:** AWS Elastic Beanstalk, Azure App Services, Google App Engine.
        - **SaaS Examples:** AWS WorkSpaces, Microsoft 365, Google Workspace.
2. Create a comparison table of the cloud service models across the service providers.

Part 3: Study of Cloud Deployment Models

1. Research and describe the four **cloud deployment models**:
    - Public Cloud
    - Private Cloud

    - Hybrid Cloud
    - Multi-Cloud
2. Discuss how AWS, Azure, and GCP enable these deployment models with examples like:
    - AWS Outposts (Hybrid Cloud)
    - Azure Stack (Private Cloud)
    - Multi-cloud integrations with Kubernetes.
3. Identify use cases for each deployment model.

Part 4: Comparative Analysis

1. Summarize the findings from Parts 1, 2, and 3 into a **comparative analysis report**.
2. Highlight the strengths, weaknesses, and unique features of AWS, Azure, and GCP in terms of their cloud architecture, service models, and deployment models.

Assignment No. 2

Title

To Study and Compare Full Virtualization and Para-Virtualization in Virtualization Technology

---

Objective

To understand the concepts of Full Virtualization and Para-Virtualization, analyze their architectures, explore their implementations, and compare their advantages, disadvantages, and use cases in real-world scenarios.

Learning Outcomes

1. Understand the working principles of Full Virtualization and Para-Virtualization.
2. Explore the role of hypervisors in virtualization technologies.
3. Compare the performance and use cases of Full Virtualization and Para-Virtualization.
4. Gain practical insights into implementing and managing virtualized environments.

Pre-requisites

- Basic knowledge of virtualization and operating systems.
- Access to virtualization platforms (e.g., VMware, Xen, KVM, VirtualBox).
- A system capable of running virtual machines.

Lab Setup

1. **Required Software:**
    - VirtualBox, VMware, or KVM for Full Virtualization.
    - Xen Hypervisor for Para-Virtualization.
2. **Hardware Requirements:**
    - A host machine with virtualization-enabled CPU.
    - Sufficient memory (8 GB or more recommended) and storage.
3. **Operating System Images:**
    - Linux distributions (e.g., Ubuntu, CentOS) or other OS images.

Assignment Tasks

Task 1: Study Full Virtualization

Steps:

1. Research and document the concept of Full Virtualization:
    - Definition and purpose.
    - Architecture and working principles.
    - Role of hypervisors (e.g., VMware ESXi, KVM).
2. Identify key features of Full Virtualization and its benefits.
3. Install and configure a virtualization platform (e.g., VirtualBox, VMware).
4. Create and run virtual machines using Full Virtualization.

Deliverables:

- A report explaining Full Virtualization architecture and features.
- Screenshots of the virtualization platform setup and running virtual machines.

Task 2: Study Para-Virtualization

Steps:

1. Research and document the concept of Para-Virtualization:
    - Definition and purpose.
    - Architecture and working principles.
    - Role of hypervisors (e.g., Xen Hypervisor).
2. Identify key features of Para-Virtualization and its benefits.
3. Install and configure the Xen Hypervisor on a host machine.
4. Create and run virtual machines using Para-Virtualization.

Deliverables:

- A report explaining Para-Virtualization architecture and features.
- Screenshots of the Xen Hypervisor setup and running virtual machines.

Task 3: Comparison of Full Virtualization and Para-Virtualization

Steps:

1. Create a comparison table for Full Virtualization and Para-Virtualization based on:
    - Architecture
    - Hypervisor types (Type-1 and Type-2)
    - Performance
    - Hardware requirements
    - Use cases
    - Advantages and disadvantages
2. Summarize the key differences and provide recommendations for use cases.

Deliverables:

- A detailed comparison table.
- Summary and recommendations.

Assignment No. 3

Title

To Implement Full Virtualization and Para-Virtualization Using Virtualization Platforms

---

Objective

To perform hands-on exercises to understand and implement Full Virtualization and Para-Virtualization using suitable virtualization platforms, analyze their performance, and document the findings.

Learning Outcomes

1. Gain practical knowledge of setting up virtual machines using Full Virtualization and Para-Virtualization.
2. Understand the configuration and operation of hypervisors.
3. Analyze and compare the performance of virtual machines created using different virtualization techniques.
4. Explore real-world applications and use cases for both approaches.

Pre-requisites

- A computer with virtualization-enabled CPU.
- Access to virtualization software (e.g., VMware, VirtualBox, KVM, Xen).
- Operating system images (e.g., Ubuntu, CentOS).
- Basic understanding of Linux commands (for Xen installation and configuration).

Lab Setup

1. **Hardware Requirements:**
    - CPU with hardware virtualization support (Intel VT-x or AMD-V).
    - Minimum 8 GB RAM and 50 GB free disk space.
2. **Required Software:**
    - VirtualBox or VMware for Full Virtualization.

    - Xen Hypervisor for Para-Virtualization.
3. **Operating Systems:**
    - Host OS: Windows/Linux.
    - Guest OS: Ubuntu Server or any Linux distribution.

Practical Tasks

Task 1: Full Virtualization Implementation

Steps:

1. Install VirtualBox or VMware Workstation on the host machine.
2. Create a virtual machine using the installed platform:
    - Assign resources (CPU, memory, storage) to the virtual machine.
    - Load the guest OS image and install it.
3. Start and configure the virtual machine:
    - Set up the network and verify connectivity.
    - Install any additional guest tools (e.g., VMware Tools or VirtualBox Guest Additions).
4. Capture screenshots of each step.

Deliverables:

- Screenshots of virtual machine setup and configuration.
- A brief report describing the steps and observations.

Task 2: Para-Virtualization Implementation

Steps:

1. Install Xen Hypervisor on the host machine:
    - Use a Linux distribution that supports Xen (e.g., CentOS, Debian).
    - Follow the official documentation to install Xen.
2. Create and configure a guest domain (DomU):
    - Allocate CPU, memory, and disk space.
    - Load the guest OS image and install it.

3. Start and configure the guest OS:
    - Verify network connectivity.
    - Enable communication between Dom0 (host domain) and DomU.
4. Capture screenshots of each step.

Deliverables:

- Screenshots of Xen Hypervisor installation and virtual machine setup.
- A brief report describing the steps and observations.

Task 3: Performance Analysis

Steps:

1. Install performance testing tools (e.g., `sysbench`, `Phoronix Test Suite`) on both Full Virtualization and Para- Virtualization VMs.
2. Run the following tests:
    - CPU performance.
    - Memory performance.
    - Disk I/O performance.
3. Record the results and compare the performance metrics.

Deliverables:

- A table summarizing the performance results.
- A brief analysis of the performance differences.

Task 4: Use Case Analysis

Steps:

1. Identify and document real-world use cases for Full Virtualization and Para-Virtualization.
2. Provide examples of scenarios where each technique would be preferred.

Deliverables:

- A list of use cases and their explanations.
- Examples of applications for each virtualization approach.

Assignment No. 4

Title

To Study and Implement Virtualization Tools - Hypervisor and Xen Architecture

---

Objective

To understand virtualization concepts and implement virtualization tools by exploring Hypervisor and Xen architecture through hands-on exercises.

Part A: Hypervisor Implementation

Step 1: Understand Virtualization and Hypervisors

- Research and define the following terms:
  - Virtualization and its benefits.
  - Types of virtualization (hardware, software, network, etc.).
  - Hypervisors (Type 1 and Type 2) with examples like VMware, Hyper-V, and VirtualBox.

Step 2: Install a Hypervisor

- Choose one hypervisor based on your system:
  - **Type 1**: VMware ESXi, Microsoft Hyper-V (if your hardware supports it).
  - **Type 2**: Oracle VirtualBox or VMware Workstation.

Step 3: Configure a Virtual Machine (VM)

1. Create a new VM using the installed hypervisor.
2. Allocate resources like CPU, RAM, disk space, and network configuration.
3. Install a guest operating system (Linux/Windows) on the VM.

Step 4: Perform Experiments

1. Launch two VMs simultaneously and perform basic tasks (e.g., file transfers or running programs).
2. Use system monitoring tools to observe CPU, memory, and disk utilization during VM operations.

Step 5: Document Observations

- Take screenshots of your hypervisor setup, VM configuration, and resource monitoring outputs.
- Record any challenges faced during the setup and how you resolved them.

Part B: Xen Architecture Implementation

Step 1: Understand Xen Architecture

- Research and document the key features of Xen:
  - Xen Hypervisor.
  - Domain 0 (Dom0) and User Domain (DomU).
  - Paravirtualization vs. Full Virtualization.

Step 2: Install Xen on Linux

1. Select a Linux distribution (e.g., Ubuntu or CentOS).
2. Install Xen hypervisor using package managers or source files.
3. Configure the bootloader (GRUB) to enable Xen as the default boot option.

Step 3: Configure and Manage Domains

1. Set up Domain 0 (Dom0) as the control domain.
2. Create and configure Domain U (DomU) as the guest domain.

Step 4: Perform Experiments

1. Launch multiple DomU instances and perform operations within them.
2. Monitor Xen's performance using tools like `xentop` or `xl list`.

Step 5: Document Observations

- Take screenshots of the Xen installation process, DomU configurations, and performance monitoring outputs.
- Record any challenges faced during the Xen setup and how they were resolved.

Part C: Comparative Analysis

Step 1: Compare Hypervisor and Xen

1. Compare the installation and configuration processes of the hypervisor and Xen.
2. Analyze performance, ease of use, resource utilization, and scalability.

Step 2: Draw Conclusions

- Discuss which tool is more suitable for specific use cases based on your observations.

Assignment No. 5

Title

To Study and Implement AWS EC2 Services

Objective

To understand the Amazon Web Services (AWS) Elastic Compute Cloud (EC2) service, including creating and managing virtual servers in the cloud. The assignment aims to provide hands-on experience in configuring, launching, and working with EC2 instances.

Steps to Study and Implement AWS EC2 Services

Part A: Understanding AWS EC2

Step 1: Research AWS EC2

- Study the key features of AWS EC2:
  - What is AWS EC2, and why is it used?
  - Types of EC2 instances (General Purpose, Compute Optimized, Memory Optimized, etc.).
  - Pricing models: On-demand, Reserved, Spot Instances, and Savings Plans.

Step 2: AWS Free Tier Setup

1. Create an AWS Free Tier account (if you do not have one).
2. Understand the free-tier eligibility limits (e.g., t2.micro instance with 750 hours per month).

Part B: Launching and Managing an EC2 Instance

Step 1: Log in to AWS Management Console

1. Navigate to the **EC2 Dashboard** under the "Compute" section.

Step 2: Launch an EC2 Instance

1. Click on **Launch Instance**.
2. Configure the instance with the following:
   - **AMI (Amazon Machine Image):** Select a free-tier eligible AMI (e.g., Amazon Linux, Ubuntu).
   - **Instance Type:** Choose `t2.micro` (free-tier eligible).
   - **Key Pair:** Create a new key pair (or use an existing one) to access the instance securely.
   - **Security Group:** Configure inbound rules to allow SSH (port 22) and HTTP (port 80) traffic.
3. Launch the instance and note the instance ID.

Step 3: Connect to the EC2 Instance

1. Use the key pair to connect via SSH:

```
bash
Copy code
ssh -i "your-key.pem" ec2-user@<Public-IP-of-Instance>
```

2. Verify the instance is running by listing files or running basic Linux commands.

Part C: Practical Experimentation

Step 1: Host a Web Server

1. Install a web server on the EC2 instance (e.g., Apache or NGINX).
   - For Apache:

```
bash
Copy code
sudo yum update -y
sudo yum install httpd -y
sudo systemctl start httpd
sudo systemctl enable httpd
```

2. Create a simple HTML file to test the server:

```
bash
Copy code
echo "<h1>Welcome to AWS EC2</h1>" | sudo tee /var/www/html/index.html
```

Step 2: Access the Web Server

1. Open a web browser and enter the public IP of the EC2 instance.
2. Verify that the HTML page loads successfully.

Part D: Advanced Configuration (Optional)

Step 1: Create and Attach an EBS Volume

1. Create an Elastic Block Store (EBS) volume from the EC2 dashboard.
2. Attach the volume to the running instance.
3. Format and mount the volume to make it usable:

```
bash
Copy code
sudo mkfs -t ext4 /dev/xvdf
sudo mkdir /data
sudo mount /dev/xvdf /data
```

Part C: Clean-Up

1. Terminate the EC2 instance to stop incurring charges.
2. Delete unused key pairs, EBS volumes, and security groups.

Assignment No. 6

Title

To Implement Auto Scaling on a Cloud Platform

Objective

The purpose of this assignment is to learn how to implement auto-scaling to dynamically adjust cloud resources based on workload demands. Students will use AWS, GCP, Azure, or an open-source cloud platform like OpenStack to configure and test auto-scaling.

Assignment Outline

Part A: Preparation

Step 1: Select a Cloud Platform

Choose one of the following platforms to complete the assignment:

- **AWS**: Auto Scaling Groups (ASG)
- **GCP**: Instance Groups with Autoscaling
- **Azure**: Virtual Machine Scale Sets (VMSS)
- **Open Source**: OpenStack Heat or Kubernetes Horizontal Pod Autoscaler (HPA)

Part B: Configuring Auto Scaling

Step 1: Environment Setup

1. **Prerequisites**:
    - Cloud account with sufficient permissions.
    - CLI tools installed for the selected platform (e.g., AWS CLI, gcloud, Azure CLI).
    - A basic application image (e.g., a simple web server).
2. **Resources to Configure**:
    - Virtual Machine or Compute Instances (based on your platform).
    - Load Balancer for distributing traffic.

Step 2: Implement Auto Scaling

AWS Auto Scaling

1. **Create a Launch Template**:
    - Define the instance type, key pair, security group, and AMI for your application.

```
bash
Copy code
aws ec2 create-launch-template --launch-template-name my-template --
version-description "v1" \
  --launch-template-data '{"InstanceType":"t2.micro","ImageId":"ami-
12345678"}'
```

2. **Set Up an Auto Scaling Group**:
    - Create an Auto Scaling Group with a minimum, maximum, and desired number of instances.

```
bash
Copy code
aws autoscaling create-auto-scaling-group --auto-scaling-group-name my-
asg \
  --launch-template LaunchTemplateName=my-template \
```

```
  --min-size 1 --max-size 5 --desired-capacity 2 \
  --vpc-zone-identifier subnet-12345678
```

3. **Configure Scaling Policies**:
   - Add scaling policies based on CPU utilization.

```
bash
Copy code
aws autoscaling put-scaling-policy --auto-scaling-group-name my-asg \
  --policy-name cpu-scale-up --policy-type TargetTrackingScaling \
  --target-tracking-configuration
'{"TargetValue":50.0,"PredefinedMetricSpecification":{"PredefinedMetric
Type":"ASGAverageCPUUtilization"}}'
```

GCP Instance Group Autoscaling

1. **Create a Managed Instance Group**:
   - Use an instance template to define the configuration.

```
bash
Copy code
gcloud compute instance-templates create my-template \
  --machine-type e2-micro --image-family ubuntu-2004-lts --image-
project ubuntu-os-cloud
gcloud compute instance-groups managed create my-instance-group \
  --template my-template --size 1 --zone us-central1-a
```

2. **Set Up Autoscaling**:
   - Enable autoscaling with a minimum and maximum number of instances.

```
bash
Copy code
gcloud compute instance-groups managed set-autoscaling my-instance-
group \
  --cool-down-period 90 --max-num-replicas 5 --min-num-replicas 1 \
  --target-cpu-utilization 0.5 --zone us-central1-a
```

Azure Virtual Machine Scale Sets

1. **Create a Scale Set**:
   - Use Azure CLI to create a scale set.

```
bash
Copy code
az vmss create --name my-scale-set --resource-group my-group \
  --image UbuntuLTS --admin-username azureuser --generate-ssh-keys \
  --instance-count 1 --vm-sku Standard_B1s
```

2. **Enable Autoscaling**:
   - Configure auto-scaling rules based on CPU usage.

```
bash
Copy code
az monitor autoscale create --resource-group my-group \
  --resource my-scale-set --resource-type
Microsoft.Compute/virtualMachineScaleSets \
```

```
  --name cpu-autoscale --min-count 1 --max-count 5 --count 2
az monitor autoscale rule create --resource-group my-group \
  --autoscale-name cpu-autoscale --condition "Percentage CPU > 70" --
scale out 1
az monitor autoscale rule create --resource-group my-group \
  --autoscale-name cpu-autoscale --condition "Percentage CPU < 30" --
scale in 1
```

OpenStack Heat Template (Optional)

1. **Write a Heat Orchestration Template**:
   - Use YAML to define the auto-scaling policy for instances.
2. **Deploy the Template**:
   - Deploy the stack using the Heat CLI.

Part C: Testing Auto Scaling

1. **Simulate High Load**:
   - Use a load-testing tool like Apache Bench (`ab`) or Locust to generate traffic.
   - Observe the scaling behavior as the load increases.
2. **Simulate Low Load**:
   - Stop the load generation and verify if the system scales down resources.
3. **Verify Resource Allocation**:
   - Check the current number of instances or resources through the dashboard or CLI.

Assignment No. 7

Title

To Implement Load Balancing on a Cloud Platform.

Objective

The objective of this assignment is to understand and implement load balancing on a cloud platform. Students will explore how to configure load balancers to distribute traffic across multiple resources to achieve better performance, scalability, and reliability. Both proprietary cloud platforms (AWS, GCP, Azure) and open-source solutions (OpenStack, HAProxy) will be considered.

Assignment Outline

Part A: Preparation

Step 1: Select a Cloud Platform

Choose one of the following platforms for the assignment:

- **AWS**: Elastic Load Balancer (ELB)
- **GCP**: Cloud Load Balancer
- **Azure**: Azure Load Balancer
- **Open Source**: HAProxy, Nginx, or OpenStack Neutron LBaaS

Step 2: Environment Setup

1. **Infrastructure Requirements**:
    - Create at least **two application servers** (VMs or containers) running a simple web application (e.g., a static HTML page or a basic Python/Node.js application).
    - Prepare a cloud environment with sufficient resources (cloud account, CLI tools, etc.).
2. **Tools/Services to Use**:
    - Web server (e.g., Apache, Nginx).
    - Load testing tools (e.g., Apache Bench, Locust).

Part B: Configuring Load Balancing

Step 1: AWS Elastic Load Balancer (ELB)

1. **Create Target Instances**:
    - Launch two or more EC2 instances and deploy a web application on each instance.
    - Ensure the security group allows HTTP/HTTPS traffic.
2. **Set Up an ELB**:
    - Use the AWS Management Console or CLI to create an ELB.

```
bash
Copy code
aws elbv2 create-load-balancer --name my-load-balancer \
  --subnets subnet-12345678 subnet-87654321 \
  --security-groups sg-12345678
```

3. **Configure Target Groups**:
    - Register the EC2 instances with the target group.

```
bash
Copy code
aws elbv2 register-targets --target-group-arn <target-group-arn> \
  --targets Id=i-12345678 Id=i-87654321
```

4. **Test the Load Balancer**:
   - Use the DNS name of the ELB to access the application.
   - Simulate traffic and observe how it distributes requests.

Step 2: GCP Cloud Load Balancer

1. **Create Instance Groups**:
   - Configure a managed instance group with your application instances.

```
bash
Copy code
gcloud compute instance-templates create web-template \
  --machine-type e2-micro --image-family debian-10 --image-project
debian-cloud
gcloud compute instance-groups managed create web-group \
  --template web-template --size 2 --zone us-central1-a
```

2. **Configure the Load Balancer**:
   - Create a GCP load balancer that forwards traffic to the instance group.

```
bash
Copy code
gcloud compute backend-services create web-backend \
  --protocol HTTP --port-name http --health-checks web-health-check \
  --global
gcloud compute url-maps create web-map --default-service web-backend
gcloud compute target-http-proxies create web-proxy --url-map web-map
gcloud compute forwarding-rules create web-rule \
  --global --target-http-proxy web-proxy --ports 80
```

Step 3: Azure Load Balancer

1. **Create Backend Resources**:
   - Launch two or more virtual machines with your application.
2. **Set Up Azure Load Balancer**:
   - Configure a load balancer in the Azure portal or CLI.

```
bash
Copy code
az network lb create --resource-group my-resource-group --name my-lb --
sku Basic \
  --backend-pool-name my-backend-pool
```

3. **Add Backend Pool and Rules**:
   - Add your VMs to the backend pool and configure load-balancing rules.

```
bash
Copy code
az network lb address-pool create --resource-group my-resource-group \
  --lb-name my-lb --name my-backend-pool
```

4. **Test Load Balancing**:
    - Access the public IP of the load balancer and observe traffic distribution.

Step 4: Open Source Load Balancing (HAProxy/Nginx)

Using HAProxy:

1. **Install HAProxy**:
    - Install HAProxy on a separate machine.

```
bash
Copy code
sudo apt-get install haproxy
```

2. **Configure HAProxy**:
    - Update the HAProxy configuration file (`/etc/haproxy/haproxy.cfg`) to include your backend servers.

```
plaintext
Copy code
frontend http_front
    bind *:80
    default_backend http_back

backend http_back
    balance roundrobin
    server server1 192.168.1.101:80 check
    server server2 192.168.1.102:80 check
```

3. **Restart HAProxy**:

```
bash
Copy code
sudo systemctl restart haproxy
```

4. **Test Load Balancing**:
    - Access the HAProxy machine's IP to check traffic distribution.

Using OpenStack Neutron LBaaS:

1. **Set Up Neutron LBaaS**:
    - Deploy instances and configure LBaaS using OpenStack CLI or Horizon dashboard.
    - Create a load balancer, listener, and pool.
2. **Assign Members to the Pool**:
    - Add your instances as pool members.
    - Test traffic distribution using the load balancer's floating IP.

Part C: Testing Load Balancing

1. **Simulate Load**:
   - Use tools like Apache Bench (`ab`) or Locust to generate traffic.

```
bash
Copy code
ab -n 1000 -c 50 http://<load-balancer-ip>/
```

2. **Observe Traffic Distribution**:
   - Check application logs or server metrics to verify how requests are distributed.
3. **Test High Availability**:
   - Stop one backend server and observe how the load balancer redirects traffic to available servers.

Assignment No. 8

Title

To Study and Implement OpenStack Services

Objective

This assignment aims to provide practical exposure to implementing and using core OpenStack services, including compute, storage, networking, and dashboard. Students will learn how to set up, configure, and utilize OpenStack services in a cloud environment.

Assignment Outline

Part A: Preparation

Step 1: Prerequisites

1. **Environment Setup**:
   - A physical or virtual machine with the following minimum configuration:
     - 8 vCPUs, 16GB RAM, 100GB storage.
     - Operating System: Ubuntu 22.04 or CentOS Stream 9.
   - Access to a machine with OpenStack pre-installed or use a tool like **DevStack**, **Packstack**, or a VirtualBox/VMWare image for easy deployment.

2. **Networking Configuration**:
   - Set up two networks: one external (for public access) and one internal (for private instances).
3. **Install OpenStack CLI**:
   - Ensure the OpenStack CLI is installed:

     ```
     bash
     Copy code
     sudo apt install python3-openstackclient
     ```

Part B: Implementing Core OpenStack Services

Step 1: Implement Compute Service (Nova)

1. **Launch a Virtual Machine**:
   - Use the OpenStack CLI or Horizon dashboard to create an instance.
   - Example command:

     ```
     bash
     Copy code
     openstack server create --image cirros --flavor m1.small \
       --key-name my-key --network demo-net my-vm
     ```
2. **Verify**:
   - Check the instance status:

     ```
     bash
     Copy code
     openstack server list
     ```
3. **Test Connectivity**:
   - Assign a floating IP to the instance for external access:

     ```
     bash
     Copy code
     openstack floating ip create public-net
     openstack server add floating ip my-vm <floating-ip>
     ```

   - Test access using SSH:

     ```
     bash
     Copy code
     ssh -i my-key.pem cirros@<floating-ip>
     ```

Step 2: Implement Storage Services (Cinder and Swift)

1. **Block Storage (Cinder)**:
   - Create a block storage volume:

```
bash
Copy code
openstack volume create --size 10 my-volume
```

- Attach the volume to an instance:

```
bash
Copy code
openstack server add volume my-vm my-volume
```

2. **Object Storage (Swift)**:
   - Create a container and upload a file:

```
bash
Copy code
openstack container create my-container
openstack object create my-container my-file.txt
```

   - List the objects in the container:

```
bash
Copy code
openstack object list my-container
```

Step 3: Implement Networking Services (Neutron)

1. **Create Networks**:
   - Internal network:

```
bash
Copy code
openstack network create private-net
openstack subnet create --network private-net --subnet-range
192.168.1.0/24 private-subnet
```

   - External network:

```
bash
Copy code
openstack network create public-net --external --provider-
network-type flat
openstack subnet create --network public-net --subnet-range
203.0.113.0/24 public-subnet
```

2. **Set Up a Router**:
   - Create and configure a router:

```
bash
Copy code
openstack router create my-router
openstack router set my-router --external-gateway public-net
openstack router add subnet my-router private-subnet
```

3. **Allocate Floating IP**:
    - Assign a floating IP to the instance as in **Step 1**.

Step 4: Use the OpenStack Dashboard (Horizon)

1. **Access the Dashboard**:
    - Open a web browser and navigate to the Horizon dashboard: `http://<controller-ip>/dashboard`.
2. **Perform Tasks via GUI**:
    - Launch an instance, create a volume, and configure networks using the dashboard.

Part C: Testing and Validation

1. **Validate Instance Launch**:
    - Ensure instances are running and accessible via floating IP.
    - Verify network configurations and connectivity.
2. **Validate Storage**:
    - Attach and mount a block storage volume to an instance.
    - Upload and retrieve objects from Swift storage.
3. **Simulate Scenarios**:
    - Test high availability by stopping one instance and observing failover.
    - Create multiple VMs to check networking and load distribution.

Assignment No. 9

Title

To Study and Implement Open Source Cloud Foundry.

Objective

This practical assignment is designed to help students understand and implement the core functionalities of Cloud Foundry, an open-source platform-as-a-service (PaaS). Students will deploy applications, integrate services, and explore the flexibility and scalability of Cloud Foundry.

Assignment Outline

Part A: Preparation

Step 1: Prerequisites

1. **Environment Setup**:
   - Use a virtual or physical machine with the following requirements:
     - Minimum: 4 vCPUs, 8GB RAM, 50GB storage.
     - Operating System: Ubuntu 22.04 or a compatible Linux distribution.
   - Install Docker and Git on your machine:

     ```
     bash
     Copy code
     sudo apt update
     sudo apt install docker.io git -y
     ```
2. **Cloud Foundry Installation**:
   - Install Cloud Foundry CLI (Command Line Interface):

     ```
     bash
     Copy code
     wget -qO-
     https://packages.cloudfoundry.org/debian/cli.cloudfoundry.org.key
     | sudo apt-key add -
     echo "deb https://packages.cloudfoundry.org/debian stable main" |
     sudo tee /etc/apt/sources.list.d/cloudfoundry-cli.list
     sudo apt update
     sudo apt install cf-cli -y
     ```

   - Clone the Cloud Foundry BOSH Lite environment for a local PaaS setup:

     ```
     bash
     Copy code
     git clone https://github.com/cloudfoundry/cf-deployment.git
     ```
3. **Deploy BOSH and Cloud Foundry**:
   - Follow the instructions in the `cf-deployment` repository to set up a local or small-scale Cloud Foundry environment.

Part B: Implementation

Step 1: Deploy a Sample Application

1. Clone a sample application repository:

   ```
   bash
   Copy code
   ```

```
git clone https://github.com/cloudfoundry-samples/spring-music.git
cd spring-music
```

2. Deploy the application to Cloud Foundry:
   - Login to your Cloud Foundry instance:

     ```
     bash
     Copy code
     cf login -a https://api.<your-cloudfoundry-instance> --skip-ssl-
     validation
     ```

   - Push the application:

     ```
     bash
     Copy code
     cf push spring-music -m 512M
     ```

3. Verify the deployment:
   - Check the status of the application:

     ```
     bash
     Copy code
     cf apps
     ```

   - Access the application using the provided URL.

Step 2: Bind a Service

1. Create a database service instance:

   ```
   bash
   Copy code
   cf create-service cleardb spark my-database
   ```

2. Bind the service to your application:

   ```
   bash
   Copy code
   cf bind-service spring-music my-database
   cf restage spring-music
   ```

3. Verify the binding:
   - Check environment variables for the app:

     ```
     bash
     Copy code
     cf env spring-music
     ```

Step 3: Scale the Application

1. Scale horizontally by increasing the number of instances:

   ```
   bash
   Copy code
   cf scale spring-music -i 3
   ```

2. Verify scaling:
   - Check the number of instances:

     ```
     bash
     Copy code
     cf apps
     ```

3. Test load distribution by accessing the application multiple times.

Step 4: Perform Application Updates

1. Modify the application code (e.g., update a welcome message).
2. Redeploy the application:

   ```
   bash
   Copy code
   cf push spring-music
   ```

3. Verify the changes in the application.

Part C: Testing and Validation

1. **Application Deployment**:
   - Confirm that the sample application is running and accessible via a public URL.
2. **Service Integration**:
   - Verify that the application is successfully connected to the database service.
3. **Scaling**:
   - Test the application's performance with multiple instances.
4. **Updates**:
   - Validate that the application reflects the deployed updates.

Conclusion:

Thus, students will have hands-on experience deploying and managing applications using Cloud Foundry.

Assignment No. 10

Title

To Study and Implement Cloud Simulators Using CloudSim.

Objective

This assignment aims to introduce students to **CloudSim**, a framework for modeling and simulation of cloud computing environments and services. Students will learn how to simulate resource allocation, task scheduling, and performance analysis in a virtual cloud environment using CloudSim.

Assignment Outline

Part A: Preparation

Step 1: Prerequisites

1. **Environment Setup**:
    - Install **Java Development Kit (JDK)** (version 8 or higher):

      ```
      bash
      Copy code
      sudo apt update
      sudo apt install openjdk-11-jdk -y
      ```

    - Install an Integrated Development Environment (IDE) such as **Eclipse** or **IntelliJ IDEA**.
    - Download and set up the **CloudSim Toolkit**:
        - Download from CloudSim GitHub.
        - Import the toolkit into your IDE as a Java project.
2. **Basic Understanding**:
    - Review the concepts of **cloud computing**, such as virtualization, data centers, and virtual machines (VMs).
    - Familiarize yourself with Java programming and object-oriented concepts.

Part B: Implementation

Step 1: Simulate a Basic Cloud Environment

1. **Setup the CloudSim Project**:

- Open the project in your IDE.
- Ensure all dependencies are correctly configured.

2. **Create the Main Simulation Class**:
   - Create a new Java class, e.g., `CloudSimExample`.
   - Import CloudSim libraries:

```
java
Copy code
import org.cloudbus.cloudsim.*;
import org.cloudbus.cloudsim.core.CloudSim;
```

3. **Initialize CloudSim**:
   - Write code to initialize the CloudSim simulation environment:

```
java
Copy code
CloudSim.init(1, Calendar.getInstance(), false);
```

4. **Create a Data Center**:
   - Define hosts, data center characteristics, and VMs.
   - Example:

```
java
Copy code
Datacenter datacenter = createDatacenter("Datacenter_0");
```

5. **Create Broker**:
   - Create a broker to manage virtual machines:

```
java
Copy code
DatacenterBroker broker = new DatacenterBroker("Broker_0");
```

6. **Run Simulation**:
   - Start the simulation:

```
java
Copy code
CloudSim.startSimulation();
```

Step 2: Implement Task Scheduling

1. **Define Cloudlets**:
   - Create a list of cloudlets (tasks) with specific parameters like length, file size, and output size.
   - Example:

```
java
Copy code
Cloudlet cloudlet = new Cloudlet(id, length, pesNumber, fileSize,
outputSize, utilizationModel);
```

2. **Assign Cloudlets to VMs**:
    - Map tasks to virtual machines using the broker.
3. **Test Different Scheduling Algorithms**:
    - Implement and compare task scheduling algorithms such as First-Come-First-Serve (FCFS) and Round Robin.

Step 3: Analyze Results

1. **Simulation Output**:
    - Analyze and print simulation results, including task completion time, resource utilization, and costs.
    - Example:

```
java
Copy code
List<Cloudlet> newList = broker.getCloudletReceivedList();
for (Cloudlet cloudlet : newList) {
    System.out.println("Cloudlet ID: " + cloudlet.getCloudletId()
+ " Status: " + cloudlet.getStatus());
}
```

2. **Plot Performance Metrics** (Optional):
    - Use external libraries like **JFreeChart** to visualize results such as resource usage and task execution time.

Part C: Testing and Validation

Step 1: Test with Different Configurations

1. Change the number of hosts, VMs, and cloudlets.
2. Modify parameters such as bandwidth, RAM, and CPU capacity.

Step 2: Validate the Results

1. Verify the correctness of task scheduling and resource allocation.
2. Compare execution times and efficiency for different scheduling algorithms.

Conclusion:

Thus, students will gain hands-on experience in simulating a cloud computing environment using CloudSim..

Assignment No. 10

Title

To Implement a Mini Project Using Cloud Platforms like AWS / GCP / Microsoft Azure / IBM Cloud / Open Source Cloud Platforms.

Objective

This practical assignment aims to guide students in designing and implementing a mini project using a cloud platform. The project will demonstrate skills in cloud service utilization, deployment, resource management, and integration of cloud-native tools and services.

Assignment Outline

Part A: Project Planning

Step 1: Choose a Cloud Platform

Select a cloud platform based on available resources or preferences:

- **AWS** (Amazon Web Services)
- **GCP** (Google Cloud Platform)
- **Microsoft Azure**
- **IBM Cloud**
- **Open Source Cloud Platforms** (e.g., OpenStack, Cloud Foundry)

Step 2: Define the Project Scope

Choose one of the following project ideas or propose a custom idea:

1. **E-Commerce Website**: Deploy a simple e-commerce web application with backend integration.
2. **Chatbot Integration**: Implement a chatbot using cloud AI/ML services.
3. **Serverless Application**: Build a serverless app using cloud functions.
4. **IoT Dashboard**: Visualize IoT sensor data in real-time on a cloud-hosted dashboard.
5. **Multi-Tier Web Application**: Deploy a multi-tier application with a database and backend API.
6. **Cloud Storage Manager**: Create a system to upload, store, and retrieve files from cloud storage.

Step 3: Define Requirements

Identify key requirements:

- Application framework (e.g., Python, Node.js, Java).
- Cloud services needed (e.g., compute, storage, databases, AI/ML).
- Security and authentication.
- Performance and scalability.

Part B: Implementation

Step 1: Setup the Environment

1. Create an account on the selected cloud platform.
2. Set up necessary SDKs, CLI tools, and IDE extensions (e.g., AWS CLI, GCP SDK, Azure CLI).
3. Familiarize yourself with the platform's web console and services.

Step 2: Design and Deploy Infrastructure

1. Use cloud services to design the infrastructure:
    - **Compute**: Provision virtual machines or serverless functions.
    - **Storage**: Set up object storage (e.g., AWS S3, Azure Blob Storage).
    - **Networking**: Configure VPCs, subnets, and load balancers.
    - **Database**: Deploy a database service (e.g., RDS, Firebase, CosmosDB).
2. Automate infrastructure provisioning using tools like **Terraform** or **CloudFormation** (optional).

Step 3: Application Deployment

1. **Code Development**: Develop the application locally.
    - Example: A Python Flask or Node.js Express app with a basic UI and database integration.
2. **Deploy to Cloud**:
    - Use a CI/CD pipeline or manual deployment.
    - Test the deployment by accessing the application through the provided cloud URL.

Step 4: Integrate Additional Services

1. **Monitoring**: Set up logging and monitoring services (e.g., AWS CloudWatch, Azure Monitor).
2. **Authentication**: Integrate a cloud-based authentication service (e.g., AWS Cognito, Firebase Authentication).
3. **AI/ML Services** (Optional): Use AI/ML APIs for features like image recognition or text analytics.

Step 5: Test and Optimize

1. Test the application's performance under different loads using cloud testing tools.
2. Optimize cloud resources for cost efficiency (e.g., spot instances, autoscaling).